

ATHENA v2: A modular Fortran framework for scientific machine learning with physics-informed, graph, and operator-based models

Ned Thaddeus Taylor¹, Harry Mclean¹, Artan Qerushi², and Steven Paul Hepplestone¹

¹ Department of Physics and Astronomy, University of Exeter, United Kingdom, EX4 4QL ² Lockheed Martin Aeronautics, Palmdale, California, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 22 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Machine learning has become an important tool across computational science, including physics, chemistry, climate science, and materials modelling. Modern network architectures such as graph neural networks (GNNs), physics-informed neural networks (PINNs), and neural operators enable models that incorporate physical structure and constraints, allowing machine learning to be applied to scientific problems such as atomistic modelling, surrogate simulation, and partial differential equation (PDE) solving.

While Python-based machine learning frameworks dominate modern development due to their flexibility and speed, they are not easily embedded into large scientific codebases written in compiled languages. This mismatch is particularly significant in computational science, most notably materials science and high-performance simulation, where Fortran remains the primary language for legacy and production systems.

ATHENA (Adaptive Training for High Efficiency Neural Network Applications) is a neural network framework implemented in modern Fortran. Since its initial release in 2024 (Taylor, 2024), ATHENA has undergone significant expansion in both scope and capability, evolving from a library focused on conventional neural networks into a broader framework for scientific machine learning. Recent developments introduce support for advanced model classes, including graph neural networks, physics-informed neural networks, recurrent architectures, and neural operators, alongside improved training workflows and model flexibility. In addition to standard dense and convolutional layers, the framework now provides native support for architectures commonly used in scientific machine learning, enabling the integration of physical constraints and structured data representations. ATHENA also introduces automated inverse design workflows through its automatic differentiation capabilities, allowing models to be used not only for prediction but also for optimisation and design tasks. The library enables users to construct, train, and deploy machine learning models directly within Fortran-based scientific applications.

The project aims to provide a flexible research platform that allows new network architectures and training workflows to be developed directly within Fortran simulation environments.

Statement of need

The growing use of machine learning in scientific computing has created demand for tools that integrate naturally with existing simulation software. While Python-based frameworks dominate the machine learning ecosystem, integrating them into large-scale Fortran simulation codes often requires external wrappers, inter-language interfaces, or separate training pipelines.

ATHENA addresses this challenge by enabling neural network development directly within Fortran. This allows machine learning models to be embedded within existing simulation codes without requiring cross-language interoperability layers. The library is particularly aimed at researchers working with large Fortran codebases in fields such as materials science, plasma physics, and computational fluid dynamics.

The software focuses on enabling experimentation with network architectures that are increasingly relevant in scientific machine learning, including:

- Graph neural networks for atomistic and structured data
- Physics-informed neural networks for solving PDEs
- Neural operators for learning mappings between functional spaces
- Inverse design for optimising input parameters to achieve desired outputs
- Multi-input and multi-output architectures for complex scientific problems

By providing these capabilities within Fortran, ATHENA allows machine learning models to be developed and deployed directly inside scientific simulation workflows.

State of the field

Several libraries provide machine learning functionality within the Fortran ecosystem. The `neural-fortran` (Curcic, 2019) provides an implementation of feed-forward and convolutional neural networks with support for parallel training. More recently, `Fiats` (Rouson et al., 2025) has utilised pure design procedures to optimise for parallel frameworks. These libraries demonstrate the feasibility of implementing neural network frameworks in Fortran. However, their primary focus is typically on efficient implementations of conventional architectures or on inference within high-performance workflows.

ATHENA differs in its emphasis on architectural flexibility and support for emerging neural network models used in scientific machine learning. In particular, the framework includes implementations of graph neural networks, neural operators, and physics-informed neural networks. These architectures are increasingly used in physics-based modelling but are rarely available within existing Fortran-based machine learning libraries.

Rather than competing directly with highly optimised deep learning frameworks, ATHENA is designed as a research-oriented platform for developing and testing new neural network methods within the Fortran ecosystem. This enables researchers to explore novel machine learning approaches directly within their existing scientific software environments.

Software design

ATHENA is designed around a modular architecture that prioritises extensibility and interoperability with scientific codes. The framework defines abstract interfaces for neural network components such as layers, activation functions, optimisers, and loss functions. New functionality can therefore be added through derived types and modular extensions without modifying the core codebase.

This design supports rapid experimentation with new architectures. For example, implementations of new graph-based layers, different neural operator functions, and custom physics-informed loss functions can be added as independent modules while maintaining compatibility with the existing training framework.

Automatic differentiation is implemented within the library via use of the `diffstruc` library to enable gradient-based optimisation across arbitrary model architectures. This can increase memory usage and computational cost, but it allows for flexible model design without requiring manual derivation of gradients. This functionality is particularly important for physics-informed

neural networks, where derivatives of the model output with respect to its inputs must be evaluated as part of the training objective.

The framework supports a range of commonly used neural network components, including dense layers, convolutional layers (1–3D), pooling layers, batch normalisation, and regularisation methods such as Dropout (Srivastava et al., 2014) and DropBlock (Ghiasi et al., 2018). Optimisation algorithms include stochastic gradient descent, RMSProp, Adam, and AdaGrad.

To support interoperability with external machine learning tools, ATHENA provides optional model serialisation using the Open Neural Network Exchange (ONNX) json-based format. Integration with experiment tracking tools such as Weights & Biases (through the external `wandb_fortran` library) further supports modern machine learning workflows within the Fortran environment.

Overall, the design emphasises flexibility and research accessibility rather than maximal runtime performance, enabling the rapid development and testing of new machine learning approaches in scientific computing contexts.

All these features are designed to be easily extended, allowing users to add new layer types, activation functions, optimisers, and other components as needed, whilst maintaining compatibility with the core training and automatic differentiation infrastructure.

The library contains a comprehensive documentation and a range of examples, which are designed to demonstrate the capabilities of the library and provide a starting point for users to implement their own models. The documentation is available online at <https://athena-fortran.readthedocs.io>, and the source code is available on GitHub at <https://github.com/nedtaylor/athena>.

Research impact statement

ATHENA is actively used in ongoing computational science research projects. One example is the RAFFLE structure prediction software (Taylor et al., 2025), where the library is being extended to utilise graph neural operators to perform inverse design of material interfaces using known local structural features.

The framework is also being used as a platform for implementing physics-informed neural networks for solving partial differential equations in plasma physics and for exploring machine learning approaches to modelling heat transport in materials via neural operators, physics-informed neural networks, and inverse design.

The project includes a growing collection of reproducible examples demonstrating these workflows, including implementations of message-passing neural networks for chemical systems, physics-informed neural networks solving Burgers' equation, and neural operators for heat transport problems.

By providing a flexible machine learning framework directly within the Fortran ecosystem, ATHENA enables researchers to incorporate modern neural network techniques into existing scientific software without requiring extensive language interoperability infrastructure.

AI usage disclosure

Development of the ATHENA software involved both direct author implementation and the use of generative AI tools.

Most of the core framework, including the training infrastructure, automatic differentiation system, and core layer implementations, was written directly by the authors. Some newer experimental layer types, including early implementations of neural operators and upcoming Kolmogorov–Arnold networks, were initially generated with assistance from generative AI tools

and subsequently reviewed, tested, and refined by the authors. Implementation of ONNX json-format model export and import functionality has been supported by generative AI tools (particularly for graph and neural operator implementations), with all generated code manually reviewed and validated by the authors to ensure correctness and consistency with the library design. Development of unit tests and examples was also supported by generative AI tools, with all generated material manually reviewed and validated by the authors.

Generative AI tools were also used to assist with drafting inline documentation and editing portions of the written documentation. All generated material was manually reviewed and validated by the authors to ensure correctness and consistency with the software implementation. Tools used for code generation and documentation include GitHub Copilot, OpenAI's ChatGPT (GPT-5.3 and 5.4), and Anthropic's Claude (Claude Sonnet 4.6 and Claude Opus 4.6).

This manuscript was written by the authors with generative AI used only for editorial assistance. ChatGPT (GPT-5.3 and 5.4) was used to assist with editing the paper text. The authors confirm that they have reviewed and approved the final version of the manuscript and that they take full responsibility for the content of the publication.

Acknowledgements

N.T.T. was supported by the Government Office for Science and the Royal Academy of Engineering under the UK Intelligence Community Postdoctoral Research Fellowships scheme (Grant No. ICRF2425-8-148). H.M. was supported by the EPSRC via Grant No. EPSRC-690010152. S.P.H. was supported by the MCC funding via Grant No. UKRI2710. A.Q. acknowledges support from Lockheed Martin Aeronautics for research on physics-inspired machine learning.

N.T.T. was the lead developer of the ATHENA software and heavily contributed to all aspects of the project, including conceptualisation, methodology, software development, validation, funding acquisition, supervision, project administration, and writing (original draft, review, and editing). H.M. contributed to the software development, conceptualisation, and validation (neural operators and inverse design), and writing (review and editing). S.P.H. contributed to funding acquisition, conceptualisation, and validation (neural operators and inverse design), and writing (review and editing). A.Q. contributed to the software development, conceptualisation, and validation (graph and physics informed neural network features), and writing (review and editing).

References

- Curcic, M. (2019). A parallel Fortran framework for neural networks and deep learning. *SIGPLAN Fortran Forum*, 38(1), 4–21. <https://doi.org/10.1145/3323057.3323059>
- Ghiasi, G., Lin, T.-Y., & Le, Q. V. (2018). DropBlock: A regularization method for convolutional networks. *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, 10750–10760. <https://dl.acm.org/doi/10.5555/3327546.3327732>
- Rouson, D., Bonachea, D., Richardson, B., Welsman, J. A., Bailey, J., Gutmann, E. D., Torres, D., Rasmussen, K., Dibba, B., Zhang, Y., Weaver, K., Bai, Z., & Nguyen, T. (2025). Fiats: Functional inference and training for surrogates. *Journal of Open Source Software*, 10(116), 8785. <https://doi.org/10.21105/joss.08785>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1), 1929–1958. <https://dl.acm.org/doi/10.5555/2627435.2670313>

- 175 Taylor, N. T. (2024). ATHENA: A fortran package for neural networks. *Journal of Open*
176 *Source Software*, 9(99), 6492. <https://doi.org/10.21105/joss.06492>
- 177 Taylor, N. T., Pitfield, J., Davies, F. H., & Hepplestone, S. P. (2025). RAFFLE: Active
178 learning accelerated interface structure prediction. *Npj Computational Materials*, 11(1).
179 <https://doi.org/10.1038/s41524-025-01749-5>

DRAFT